



Email Spam Detection System Using Machine Learning and Natural Language Processing

Internship Project Report

Submitted By

Sudhanshu Ranjan

Khushwinder Kaur

Internship Organization

Solitaire Infosystems Pvt. Ltd.

Academic Session 2025-2026



Certificate

This is to certify that the project titled "Email Spam Detection System Using Machine Learning and Natural Language Processing" has been successfully completed by Sudhanshu Ranjan and Khushwinder Kaur during their internship at Solitaire Infosystems Pvt. Ltd.

The project has been developed under proper supervision and guidance. The work carried out by the students demonstrates their understanding of Machine Learning, Natural Language Processing, Data Analysis, and Web Application Development using Python and Streamlit.

The project fulfills the objectives assigned during the internship period and is found satisfactory for submission. The students have shown dedication, technical competence, and problem-solving ability throughout the development of this project.

Project Guide

Solitaire Infosystems Pvt. Ltd.

Date: _____

Place: _____

Declaration

We hereby declare that the project report entitled "Email Spam Detection System Using Machine Learning and Natural Language Processing" submitted as part of our internship at Solitaire Infosystems Pvt. Ltd. is an original piece of work carried out by us.

This project has been developed independently under the guidance of our mentors using publicly available datasets and open-source technologies. The work presented in this report has not been submitted previously, either in part or in full, to any university, institute, or organization for the award of any degree, diploma, or certification.

All references, documentation, research papers, datasets, and online resources used during the development of this project have been properly acknowledged in the references section.

Submitted By

Sudhanshu Ranjan

Khushwinder Kaur

Date: _____

Acknowledgement

We express our sincere gratitude to Solitaire Infosystems Pvt. Ltd. for providing us with the opportunity to undergo industrial training and gain practical exposure to Machine Learning, Natural Language Processing, and web application development.

We are thankful to our project mentors for their continuous guidance, constructive feedback, and encouragement throughout the project lifecycle. Their support helped us understand the complete machine learning workflow, from dataset preparation and model training to evaluation and deployment.

We also thank our faculty members for their academic support, and the open-source community behind Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, NLTK, Plotly, Streamlit, and Joblib for making modern machine learning development accessible and efficient.

Finally, we thank our family members and friends for their encouragement and motivation during the internship period.

Abstract

Electronic mail is one of the most widely used communication channels for personal, academic, business, banking, healthcare, and government communication. The increasing use of email has also led to a sharp rise in spam messages, phishing attempts, fraudulent offers, malware links, and other unwanted content.

Traditional spam filters depend mainly on manually written rules, blacklists, and keyword matching. These approaches are difficult to maintain because modern spam messages constantly change their wording, structure, and delivery patterns. Machine Learning provides a more adaptive solution by learning patterns from historical labeled data.

This project develops an intelligent Email Spam Detection System using Natural Language Processing and supervised machine learning. Raw email text is cleaned, normalized, tokenized, filtered through stopword removal, and transformed into numerical features using TF-IDF Vectorization. A Linear Support Vector Classifier is then trained to classify messages as Spam or Ham.

The trained model is integrated into a Streamlit web application that allows users to enter email content and receive instant predictions. The project demonstrates an end-to-end machine learning pipeline covering data preparation, exploratory analysis, feature engineering, model training, evaluation, testing, and deployment.

Keywords: Machine Learning, Natural Language Processing, Email Spam Detection, TF-IDF, LinearSVC, Streamlit, Python, Scikit-learn, Text Classification

Table of Contents

Certificate	1
Declaration	2
Acknowledgement	3
Abstract	4
Table of Contents	5
List of Figures	9
List of Tables	10
Chapter 1 - Introduction	11
1.1 Introduction	11
1.2 Background of the Study	11
1.3 Need for the Project	11
1.4 Problem Statement	11
1.5 Objectives of the Project	12
1.6 Scope and Applications	12
Chapter 2 - Company Profile	12
2.1 Introduction to Solitaire Infosystems Pvt. Ltd.	12
2.2 Vision	12
2.3 Mission	13
2.4 Services Offered	13
Chapter 3 - Literature Review	13
3.1 Introduction	13
3.2 Rule-Based Spam Filtering	13
3.3 Machine Learning-Based Spam Detection	14
3.4 NLP in Spam Detection	14
3.5 Comparison of Common Algorithms	14
3.6 Why LinearSVC Was Selected	14
Chapter 4 - Existing System	14
4.1 Existing Email Filtering Techniques	14
4.2 Limitations of Existing Systems	15
Chapter 5 - Proposed System	15
5.1 Overview	15
5.2 System Workflow	15
5.3 Architecture Explanation	16
5.4 Advantages of the Proposed System	16
Chapter 6 - Technology Stack	16
6.1 Introduction	16
6.2 Libraries and Tools Used	17
Chapter 7 - System Requirements	17

7.1 Software Requirements	17
7.2 Hardware Requirements	18
Chapter 8 - Dataset Description	18
8.1 Introduction	18
8.2 Dataset Overview	18
8.3 Features Used	18
8.4 Data Collection and Cleaning	19
Chapter 9 - Exploratory Data Analysis	19
9.1 Introduction	19
9.2 Objectives of EDA	19
9.3 Class Distribution	20
9.4 Message Length Analysis	20
9.5 Word Frequency Analysis	20
9.6 Correlation Analysis	21
Chapter 10 - Natural Language Processing	22
10.1 Introduction	22
10.2 Text Normalization	22
10.3 Punctuation and Noise Removal	22
10.4 Tokenization	22
10.5 Stopword Removal	22
10.6 Stemming	22
Chapter 11 - Feature Engineering	23
11.1 Introduction	23
11.2 Feature Engineering Steps	23
11.3 Importance of Feature Engineering	23
Chapter 12 - TF-IDF Vectorization	24
12.1 Introduction	24
12.2 Term Frequency	24
12.3 Inverse Document Frequency	24
12.4 Final TF-IDF Formula	24
12.5 Example	24
12.6 Advantages of TF-IDF	24
Chapter 13 - Machine Learning Model (LinearSVC)	25
13.1 Introduction	25
13.2 Working Principle	25
13.3 Training Process	25
13.4 Hyperparameters	25
13.5 Model Saving	26
Chapter 14 - Model Evaluation	26
14.1 Introduction	26

14.2 Evaluation Metrics	26
14.3 Confusion Matrix	26
14.4 Classification Report	27
14.5 Performance Analysis	27
Chapter 15 - Streamlit Application	27
15.1 Introduction	27
15.2 Application Features	28
15.3 Working of the Application	29
Chapter 16 - Testing	29
16.1 Introduction	29
16.2 Test Cases	29
16.3 Testing Summary	30
Chapter 17 - Advantages & Limitations	30
17.1 Advantages	30
17.2 Limitations	30
Chapter 18 - Conclusion	30
Chapter 19 - Future Scope	31
References	32
Appendix	33
Appendix A - Software Used	33
Appendix B - Python Libraries	33
Appendix C - Screenshot Checklist	33
Appendix D - Core Prediction Pipeline	34
Internship Learning Outcomes	34

List of Figures

Figure 5.1: Proposed System Workflow	16
Figure 9.1: Spam vs Ham Distribution	20
Figure 9.2: Message Length Distribution	20
Figure 9.3: Most Frequent Words in Spam Messages	21
Figure 9.4: Most Frequent Words in Ham Messages	21
Figure 9.5: Correlation Heatmap	21
Figure 14.1: Confusion Matrix	27
Figure 15.1: Streamlit Home Page	28
Figure 15.2: Streamlit Prediction Interface	28
Figure 15.3: Streamlit Dashboard	29

List of Tables

Table 2.1: Services Offered by Solitaire Infosystems Pvt. Ltd.	13
Table 3.1: Comparison of Common Spam Classification Algorithms	14
Table 4.1: Limitations of Traditional Spam Filtering	15
Table 6.1: Technology Stack Summary	17
Table 7.1: Software Requirements	17
Table 7.2: Hardware Requirements	18
Table 8.1: Dataset Overview	18
Table 8.2: Dataset Attributes	18
Table 9.1: Dataset Characteristics Observed During EDA	19
Table 10.1: Stemming Examples	22
Table 11.1: Feature Engineering Benefits	23
Table 12.1: Example of Preprocessed Emails	24
Table 13.1: Train-Test Split	25
Table 13.2: Key LinearSVC Hyperparameters	25
Table 14.1: Confusion Matrix Interpretation	26
Table 14.2: Classification Report	27
Table 15.1: Streamlit Application Features	28
Table 16.1: Functional Test Cases	29
Code Block A.1: Example Scikit-learn Pipeline	34

Chapter 1 - Introduction

1.1 Introduction

Email has become a critical communication medium for individuals, organizations, educational institutions, government agencies, and businesses. It enables fast and low-cost communication, but its popularity has also made it a common channel for unsolicited and malicious messages.

Spam emails include unwanted advertisements, phishing messages, fake lottery notifications, fraudulent investment offers, malware links, and identity theft attempts. Apart from wasting time and storage, spam can compromise sensitive personal and financial information.

The proposed system combines Natural Language Processing with Machine Learning to classify incoming email text as Spam or Ham. Text is preprocessed, converted into TF-IDF features, and classified using a Linear Support Vector Classifier. A Streamlit interface makes the trained model accessible to users in real time.

1.2 Background of the Study

Early spam filters relied on static rules, keyword lists, and sender blacklists. These methods work only when spam patterns are known in advance. Modern spammers often modify words, use symbols, alter URLs, or change message structure to bypass static filters.

Machine learning models can learn recurring patterns from labeled examples and classify unseen messages more effectively. NLP techniques further improve performance by cleaning text and extracting meaningful linguistic features.

1.3 Need for the Project

- Protection from phishing, malware, and fraudulent links.
- Reduction in manual effort spent deleting unwanted emails.
- Improved productivity and cleaner inbox management.
- Lower risk of financial loss caused by email-based scams.
- A scalable approach that can learn from new data over time.

1.4 Problem Statement

The rapid increase in unsolicited emails has created serious challenges for users and email service providers. Conventional filters require continuous manual updates and often fail to

detect new spam patterns. The project therefore aims to develop an intelligent spam detection system that automatically identifies spam messages with high accuracy while minimizing false positives and false negatives.

1.5 Objectives of the Project

- Collect and prepare a reliable spam and ham message dataset.
- Preprocess raw text using NLP techniques.
- Convert text into numerical features using TF-IDF Vectorization.
- Train a supervised machine learning classifier for spam detection.
- Evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
- Deploy the trained model through an interactive Streamlit application.

1.6 Scope and Applications

The scope of the project includes dataset cleaning, NLP preprocessing, feature extraction, model training, model evaluation, and Streamlit deployment. The system can be applied by email service providers, banks, educational institutions, government offices, corporate organizations, healthcare providers, and e-commerce platforms to reduce email-based security risks.

Chapter 2 - Company Profile

2.1 Introduction to Solitaire Infosystems Pvt. Ltd.

Solitaire Infosystems Pvt. Ltd. is an Information Technology company that provides services in software development, Artificial Intelligence, Machine Learning, Data Science, Cloud Computing, Mobile Application Development, and Web Technologies. The organization focuses on practical training, live projects, and industry-oriented learning.

During the internship, the project team worked on the Email Spam Detection System and gained hands-on experience in data preprocessing, NLP, machine learning model development, evaluation, and deployment using Streamlit.

2.2 Vision

The vision of the organization is to deliver innovative, scalable, and intelligent software solutions while supporting young professionals through practical industry exposure.

2.3 Mission

- Provide high-quality software solutions using emerging technologies.
- Encourage innovation through research and development.
- Offer industry-oriented training programs for students.
- Develop skilled professionals capable of solving real-world problems.
- Maintain strong standards of software quality and customer satisfaction.

2.4 Services Offered

Table 2.1: Services Offered by Solitaire Infosystems Pvt. Ltd.

Service Area	Description
Artificial Intelligence and Machine Learning	Development of intelligent systems, predictive models, and automation solutions.
Data Science	Data analysis, dashboards, statistical analysis, and business intelligence solutions.
Web Development	Responsive and secure web applications using modern web technologies.
Mobile Application Development	Android and cross-platform applications for business requirements.
Cloud Computing	Deployment and cloud-based solutions using platforms such as AWS, Azure, and Google Cloud.
Internship and Training	Practical exposure to current technologies and software development practices.

Chapter 3 - Literature Review

3.1 Introduction

Email spam detection is an important research area in Artificial Intelligence and Cybersecurity. Earlier approaches focused on manual filtering rules, while modern approaches rely on NLP and machine learning to learn patterns from labeled messages.

3.2 Rule-Based Spam Filtering

Rule-based systems classify emails using predefined keywords, sender addresses, blacklisted domains, or suspicious phrases such as free money, congratulations, winner, or click here. These systems are easy to implement but require continuous updates and can be bypassed by slight text modifications.

3.3 Machine Learning-Based Spam Detection

Machine learning introduced a data-driven approach. Instead of manually defining every rule, the system learns from historical examples, extracts meaningful patterns, and predicts whether new messages are spam or ham.

3.4 NLP in Spam Detection

NLP converts unstructured email text into a clean representation suitable for algorithms. Important preprocessing steps include lowercase conversion, punctuation removal, tokenization, stopword removal, stemming, and feature extraction.

3.5 Comparison of Common Algorithms

Table 3.1: Comparison of Common Spam Classification Algorithms

Algorithm	Strengths	Limitations
Naive Bayes	Fast, simple, effective on small text datasets.	Assumes feature independence.
Logistic Regression	Easy to interpret and efficient.	May underperform on complex text boundaries.
Decision Tree	Simple decision rules and visual explanation.	Prone to overfitting.
Random Forest	Improves accuracy and reduces overfitting.	Larger model size and higher computation.
Support Vector Machine	Strong performance on sparse, high-dimensional TF-IDF data.	Training can be slower on extremely large datasets.

3.6 Why LinearSVC Was Selected

- Performs well on high-dimensional text classification tasks.

- Handles sparse TF-IDF matrices efficiently.
- Offers strong classification accuracy for binary problems.
- Trains faster than kernel-based SVM models on large feature spaces.
- Consumes less memory than many complex ensemble methods.

Chapter 4 - Existing System

4.1 Existing Email Filtering Techniques

Traditional email systems commonly use keyword matching, sender blacklists, domain filters, manual rules, and basic heuristic checks. When a message satisfies one or more predefined rules, it is classified as spam.

4.2 Limitations of Existing Systems

Table 4.1: Limitations of Traditional Spam Filtering

Limitation	Impact
High maintenance	Rules must be updated continuously as spam vocabulary changes.
Poor adaptability	New and disguised spam patterns may bypass static filters.
False positives	Legitimate emails can be incorrectly sent to the spam folder.
False negatives	Spam emails may reach users when rules fail to match.
Manual configuration	Requires expert intervention and frequent tuning.
Limited intelligence	Does not understand textual context or learned patterns.

Chapter 5 - Proposed System

5.1 Overview

The proposed system uses Natural Language Processing and supervised Machine Learning to automatically classify messages as Spam or Ham. The model learns from labeled examples rather than relying on manually configured rules.

5.2 System Workflow

The workflow begins with dataset collection and cleaning. Text is then normalized and converted into numerical TF-IDF vectors. The LinearSVC model learns the relationship between these vectors and their labels. After evaluation, the trained model is saved and connected to the Streamlit application for real-time prediction.

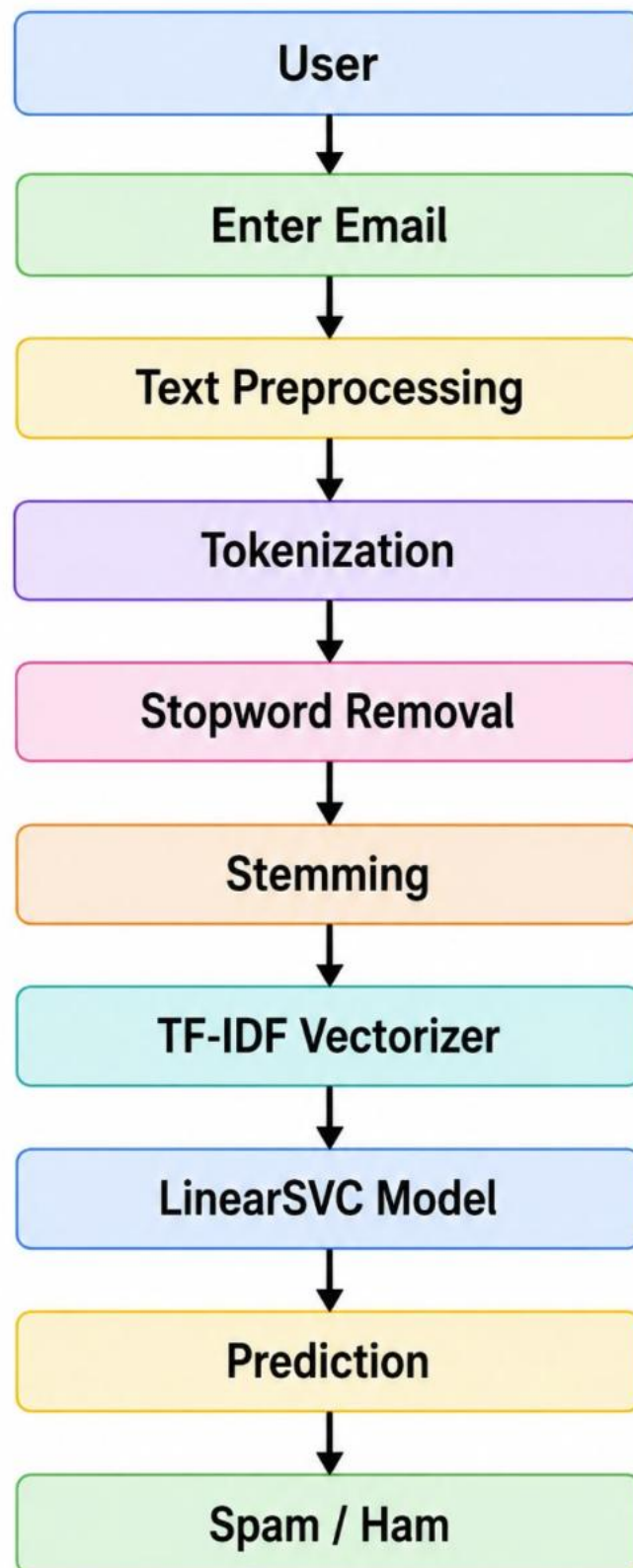


Figure 5.1: Proposed System Workflow

5.3 Architecture Explanation

- User enters an email message through the Streamlit interface.
- The input text is cleaned by removing noise such as punctuation, URLs, extra spaces, and symbols.
- Tokens are filtered through stopword removal and stemming.
- TF-IDF Vectorization converts the cleaned message into a numerical feature vector.
- The trained LinearSVC model predicts whether the message is spam or ham.
- The result is displayed instantly to the user.

5.4 Advantages of the Proposed System

- Higher accuracy than static keyword-based filters.
- Automatic learning from historical examples.
- Fast prediction suitable for real-time use.
- Scalable architecture based on open-source technologies.
- User-friendly interface through Streamlit.
- Improved cybersecurity by reducing exposure to phishing and fraud.

Chapter 6 - Technology Stack

6.1 Introduction

The project required a combination of programming tools, data analysis libraries, machine learning frameworks, visualization packages, and deployment technologies. Python was selected because of its mature ecosystem for NLP and machine learning.

6.2 Libraries and Tools Used

Table 6.1: Technology Stack Summary

Component	Technology Used	Purpose
Programming Language	Python	Core development and model implementation.
IDE	Jupyter Notebook, VS Code	Exploration, development, and application coding.

Data Handling	Pandas	Reading CSV files, cleaning, filtering, and transformation.
Numerical Computing	NumPy	Array and matrix operations.
Visualization	Matplotlib, Seaborn, Plotly	EDA charts and performance visualizations.
NLP	NLTK	Tokenization, stopword removal, stemming, and normalization.
Machine Learning	Scikit-learn	TF-IDF Vectorization, train-test split, LinearSVC, metrics.
Deployment	Streamlit	Interactive web interface for predictions.
Model Storage	Joblib	Saving and loading trained pipelines.
Version Control	Git and GitHub	Source code management and collaboration.

Chapter 7 - System Requirements

7.1 Software Requirements

Table 7.1: Software Requirements

Software	Purpose
Windows 10/11 or macOS	Operating system for development and execution.
Python 3.11+	Programming language and runtime environment.
Jupyter Notebook	Model development and exploratory analysis.
Visual Studio Code	Streamlit application development.
Streamlit	Local and web-based deployment interface.
Git	Version control.
GitHub	Source code hosting and project backup.

7.2 Hardware Requirements

Table 7.2: Hardware Requirements

Hardware	Minimum Requirement
Processor	Intel Core i3 or higher.
RAM	4 GB minimum; 8 GB recommended.
Storage	2 GB free space.
Internet	Required for package installation and deployment.
Display	1366 x 768 resolution or higher.

Chapter 8 - Dataset Description

8.1 Introduction

Machine learning models depend strongly on the quality and representativeness of the dataset. For this project, publicly available spam datasets containing labeled SMS and email messages were used. The data includes both spam and legitimate messages.

8.2 Dataset Overview

Table 8.1: Dataset Overview

Property	Description
Data format	CSV
Problem type	Binary text classification
Target classes	Spam and Ham
Input feature	Message text
Output feature	Label
Language	English
Sources	Public spam message collections used for academic machine learning projects.

8.3 Features Used

Table 8.2: Dataset Attributes

Attribute	Description
Label	Target class indicating whether the message is Spam or Ham.
Message	Actual email or SMS text used as the input feature after preprocessing.

8.4 Data Collection and Cleaning

- Collected publicly available labeled spam and ham messages.
- Removed duplicate records and null entries.
- Standardized labels to a consistent Spam/Ham format.
- Converted text to lowercase.
- Removed punctuation, numbers where appropriate, extra whitespace, special symbols, and unwanted characters.

Chapter 9 - Exploratory Data Analysis

9.1 Introduction

Exploratory Data Analysis helps understand the structure, quality, and patterns in a dataset before model training. In this project, EDA was used to inspect class distribution, message length, missing values, duplicate records, word frequency, and relationships between derived text features.

9.2 Objectives of EDA

- Understand the number of records and columns.
- Identify missing values and duplicate messages.
- Study the distribution of spam and ham classes.
- Analyze message length and word count.
- Identify frequently occurring words in each class.
- Prepare the dataset for feature engineering and model training.

Table 9.1: Dataset Characteristics Observed During EDA

Property	Observation
Dataset type	Text dataset suitable for binary classification.
Classes	Spam and Ham.
Missing values	No significant missing values were found after inspection.
Duplicates	Duplicate entries were removed to reduce bias.
Class balance	Ham messages were more frequent than spam messages, which reflects real-world email usage.

9.3 Class Distribution

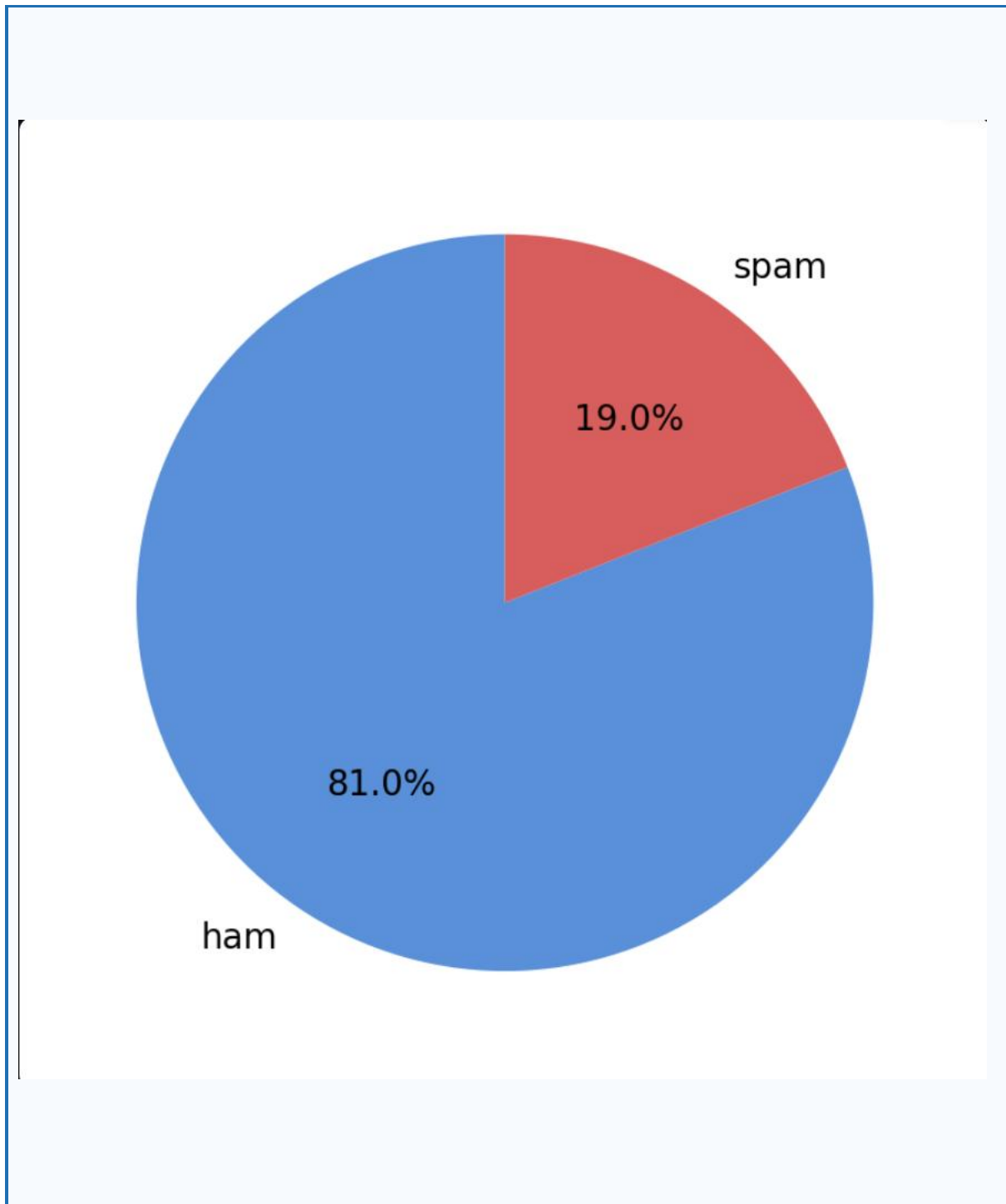


Figure 9.1: Spam vs Ham Distribution

The class distribution showed that ham messages formed the majority of the dataset, while spam messages represented a smaller but sufficient portion for model training.

9.4 Message Length Analysis

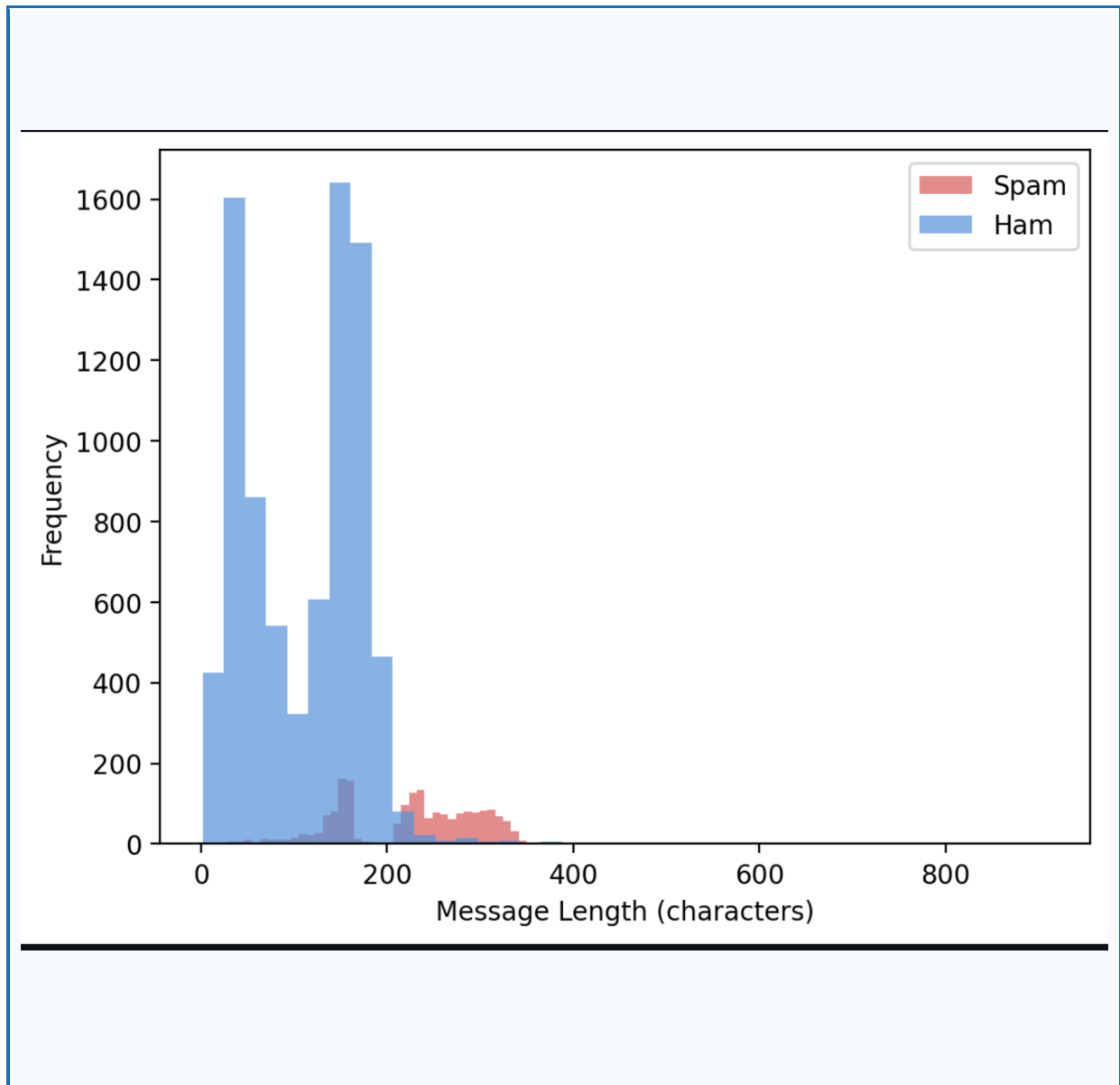


Figure 9.2: Message Length Distribution

Message length was analyzed because spam messages often contain repeated promotional language, links, and urgent calls to action. Although message length alone is not enough for reliable classification, it provides useful supporting insight.

9.5 Word Frequency Analysis

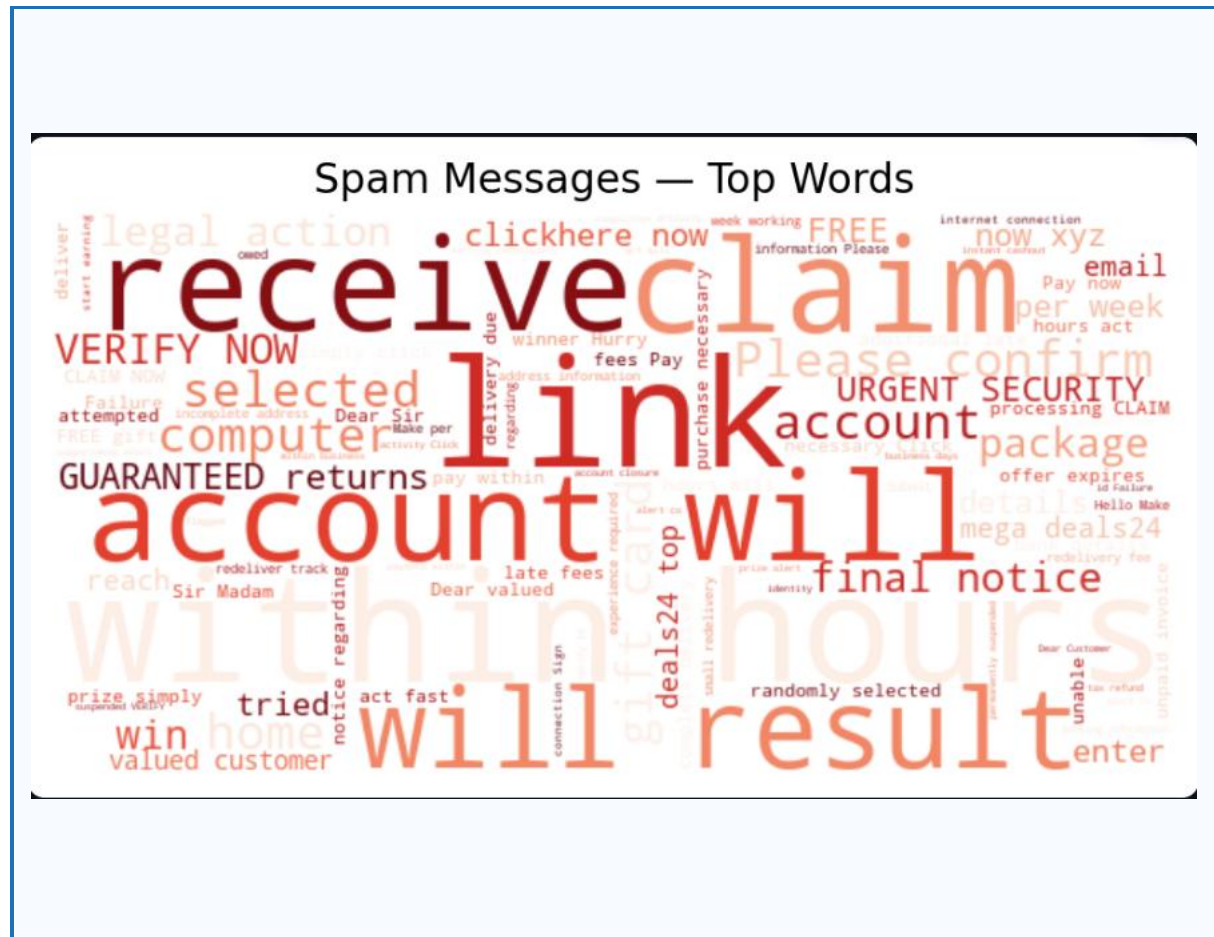


Figure 9.3: Most Frequent Words in Spam Messages



Figure 9.4: Most Frequent Words in Ham Messages

Spam messages commonly contained words such as free, win, prize, offer, cash, claim, and urgent. Ham messages more often contained conversational and work-related words such as meeting, thanks, project, family, office, and tomorrow.

9.6 Correlation Analysis

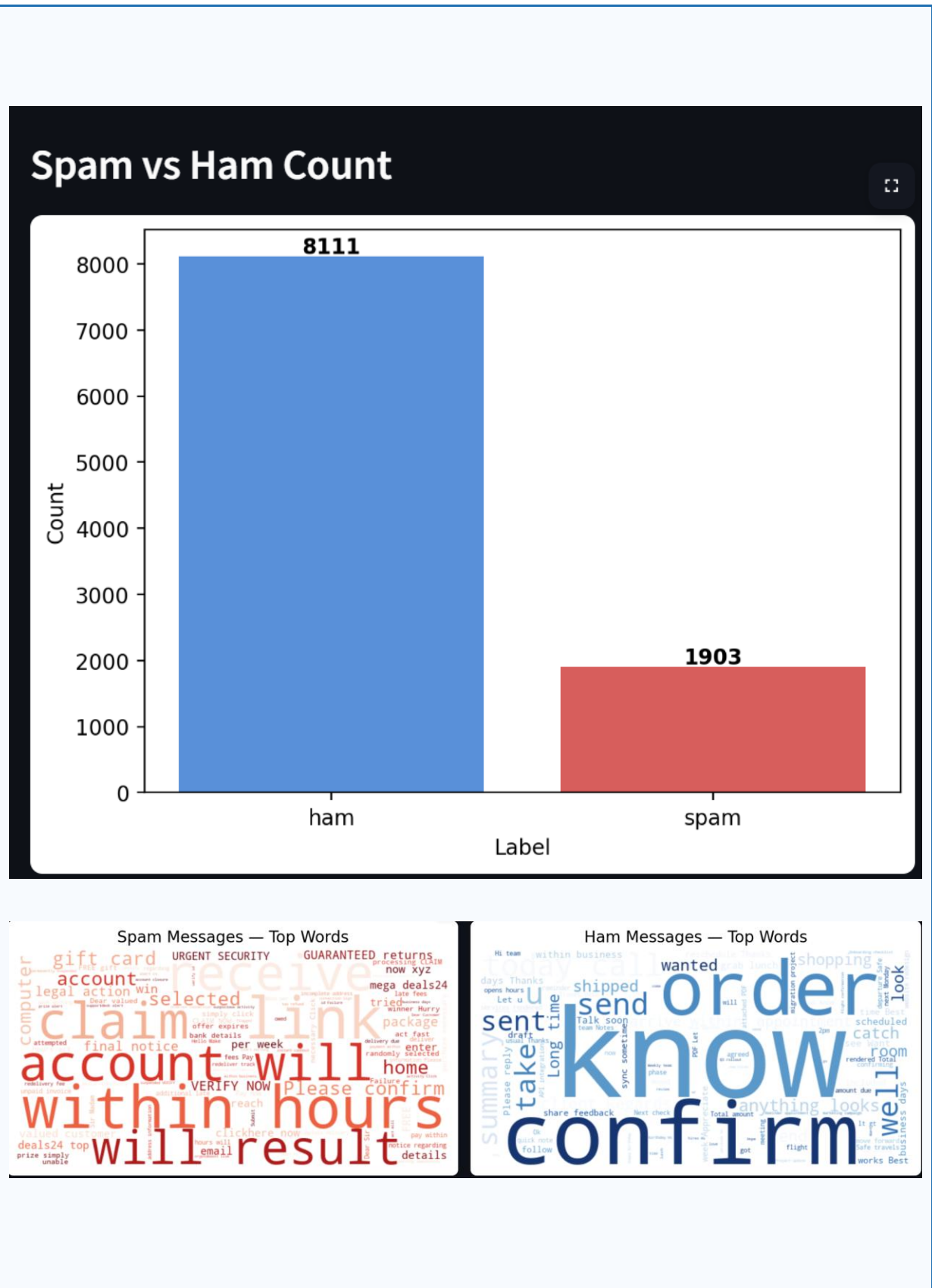


Figure 9.5: Correlation Heatmap

Correlation analysis was performed on derived numerical features such as message length and word count. These features showed expected relationships, but TF-IDF features were still required for effective spam classification.

Chapter 10 - Natural Language Processing

10.1 Introduction

Natural Language Processing enables computers to process and analyze human language. Since email messages are unstructured text, NLP is essential for transforming raw messages into a clean and consistent form that a machine learning model can use.

10.2 Text Normalization

Text normalization converts input text into a standard format. In this project, all messages were converted to lowercase, unnecessary spaces were removed, and text format was standardized.

10.3 Punctuation and Noise Removal

Punctuation marks, unwanted symbols, extra spaces, URLs, and irrelevant characters were removed because they often increase vocabulary size without improving classification quality.

10.4 Tokenization

Tokenization breaks a sentence into individual words or tokens. For example, the sentence "You have won a free lottery" becomes the tokens you, have, won, a, free, and lottery.

10.5 Stopword Removal

Stopwords such as the, is, are, and, in, on, and for appear frequently in both spam and ham messages. Removing them reduces dimensionality and helps the model focus on more informative words.

10.6 Stemming

Table 10.1: Stemming Examples

Original Word	Stemmed Word
Playing	Play

Played	Play
Plays	Play
Winner	Win
Winning	Win
Offering	Offer

Stemming reduces related words to a common root form, helping the model generalize across different word variations.

Chapter 11 - Feature Engineering

11.1 Introduction

Feature Engineering is the process of transforming raw data into meaningful input variables for machine learning algorithms. In text classification, this step is critical because algorithms cannot directly interpret raw sentences.

11.2 Feature Engineering Steps

- Clean the raw email text.
- Tokenize the text into individual words.
- Remove stopwords and unnecessary symbols.
- Apply stemming to reduce vocabulary size.
- Transform the cleaned text into numerical features using TF-IDF Vectorization.

11.3 Importance of Feature Engineering

Well-engineered features improve prediction accuracy, reduce computational complexity, remove irrelevant information, lower the risk of overfitting, and help the classifier learn the most useful differences between spam and ham messages.

Table 11.1: Feature Engineering Benefits

Benefit	Explanation
Improved accuracy	Noise removal helps the model focus on useful words.

Reduced dimensionality	Stopword removal and stemming reduce vocabulary size.
Faster training	Cleaner input reduces unnecessary computation.
Better generalization	Related word forms are grouped through stemming.

Chapter 12 - TF-IDF Vectorization

12.1 Introduction

TF-IDF, or Term Frequency-Inverse Document Frequency, is a feature extraction technique used to convert text into numerical vectors. It assigns higher scores to words that are important in a message but not common across every message.

12.2 Term Frequency

Term Frequency measures how often a word appears in a document relative to the total number of words in that document.

$$TF = (\text{Number of times a term appears in a document}) / (\text{Total number of terms in the document}) \quad (12.1)$$

12.3 Inverse Document Frequency

Inverse Document Frequency measures how rare or unique a word is across the entire document collection.

$$IDF = \log(\text{Total number of documents} / \text{Number of documents containing the term}) \quad (12.2)$$

12.4 Final TF-IDF Formula

$$TF-IDF = TF \times IDF \quad (12.3)$$

Words such as lottery, prize, claim, free, and urgent often receive higher weights because they are more informative for spam classification. Very common words receive lower weights because they appear in many documents.

12.5 Example

Table 12.1: Example of Preprocessed Emails

Email	Preprocessed Text
Email 1	congratulations won free lottery
Email 2	attend project meeting tomorrow

12.6 Advantages of TF-IDF

- Captures the importance of words within individual messages.
- Reduces the influence of common words.
- Handles large vocabularies efficiently.
- Produces sparse but informative vectors.
- Works well for spam filtering, sentiment analysis, and document classification.

Chapter 13 - Machine Learning Model (LinearSVC)

13.1 Introduction

After TF-IDF Vectorization, the next step is to train a classifier capable of separating spam messages from legitimate messages. The Linear Support Vector Classifier was selected because it performs well on high-dimensional sparse text features.

13.2 Working Principle

LinearSVC attempts to find the optimal linear decision boundary, called a hyperplane, that separates spam and ham examples. The model maximizes the margin between classes so that it can generalize effectively to unseen messages.

13.3 Training Process

- Load the cleaned dataset.
- Separate message text and class labels.
- Split the dataset into training and testing subsets.
- Transform text into TF-IDF vectors.
- Train the LinearSVC classifier.
- Evaluate performance on the testing set.
- Save the trained model using Joblib.

Table 13.1: Train-Test Split

Dataset Portion	Percentage
Training Data	80%
Testing Data	20%

13.4 Hyperparameters

Table 13.2: Key LinearSVC Hyperparameters

Parameter	Purpose
C	Controls the trade-off between margin size and classification error.
max_iter	Defines the maximum optimization iterations.
loss	Specifies the loss function used during optimization.
random_state	Supports reproducible experiments.

13.5 Model Saving

The trained model pipeline was saved with Joblib so that the Streamlit application can load it directly during prediction. This avoids retraining the model each time the application starts.

Chapter 14 - Model Evaluation

14.1 Introduction

Model evaluation determines whether the trained classifier performs reliably on unseen data. Accuracy alone may be misleading when class distribution is not perfectly balanced, so multiple metrics were used.

14.2 Evaluation Metrics

Accuracy measures the proportion of correctly classified messages.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \quad (14.1)$$

Precision measures how many messages predicted as spam are actually spam.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad (14.2)$$

Recall measures how many actual spam messages were correctly detected.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \quad (14.3)$$

F1-score is the harmonic mean of precision and recall.

$$\text{F1} = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (14.4)$$

14.3 Confusion Matrix

Table 14.1: Confusion Matrix Interpretation

Actual / Predicted	Ham	Spam
Ham	True Negative (TN)	False Positive (FP)
Spam	False Negative (FN)	True Positive (TP)

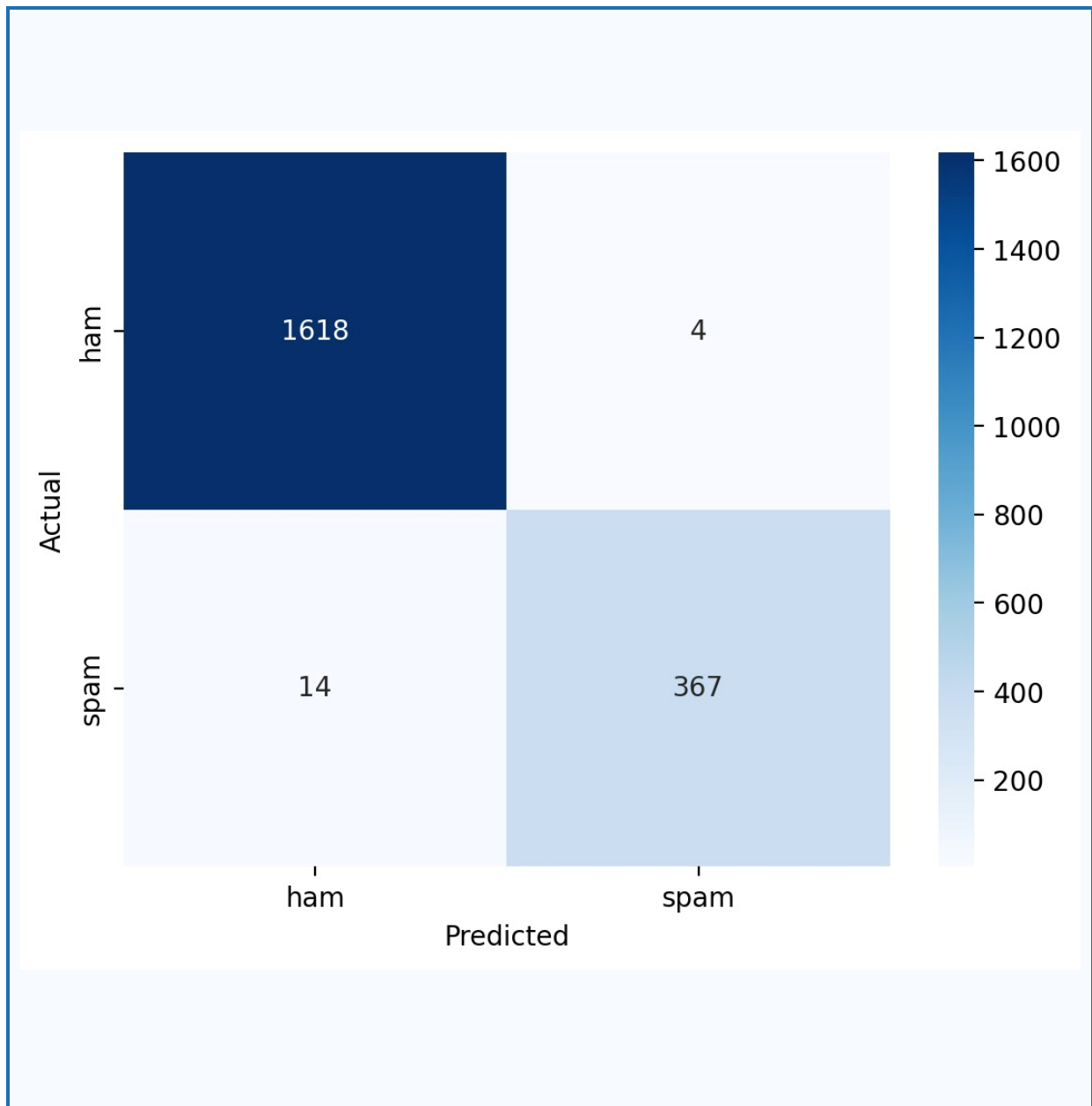


Figure 14.1: Confusion Matrix

14.4 Classification Report

Table 14.2: Classification Report

Metric	Ham	Spam
Precision	99.14%	98.92%
Recall	99.75%	96.33%
F1-Score	99.45%	97.61%
Support	1622	381

14.5 Performance Analysis

The LinearSVC model achieved strong performance in classifying spam emails. TF-IDF features captured meaningful word patterns, while the classifier separated spam and ham messages with high precision and recall. The results indicate that the system is suitable for practical spam filtering scenarios, while still allowing future retraining as spam patterns evolve.

Chapter 15 - Streamlit Application

15.1 Introduction

The trained model was deployed using Streamlit so that users can interact with the spam detector through a simple web interface. Users enter email text, click the prediction button, and instantly receive the classification result.

15.2 Application Features

Table 15.1: Streamlit Application Features

Feature	Description
Home page	Introduces the project and purpose of the application.
Email input area	Allows users to paste or type an email message.
Prediction button	Triggers preprocessing, vectorization, and model prediction.
Result display	Shows whether the email is Spam or Ham.
Dashboard	Displays dataset and model visualizations.
About section	Describes project scope, tools, and model workflow.

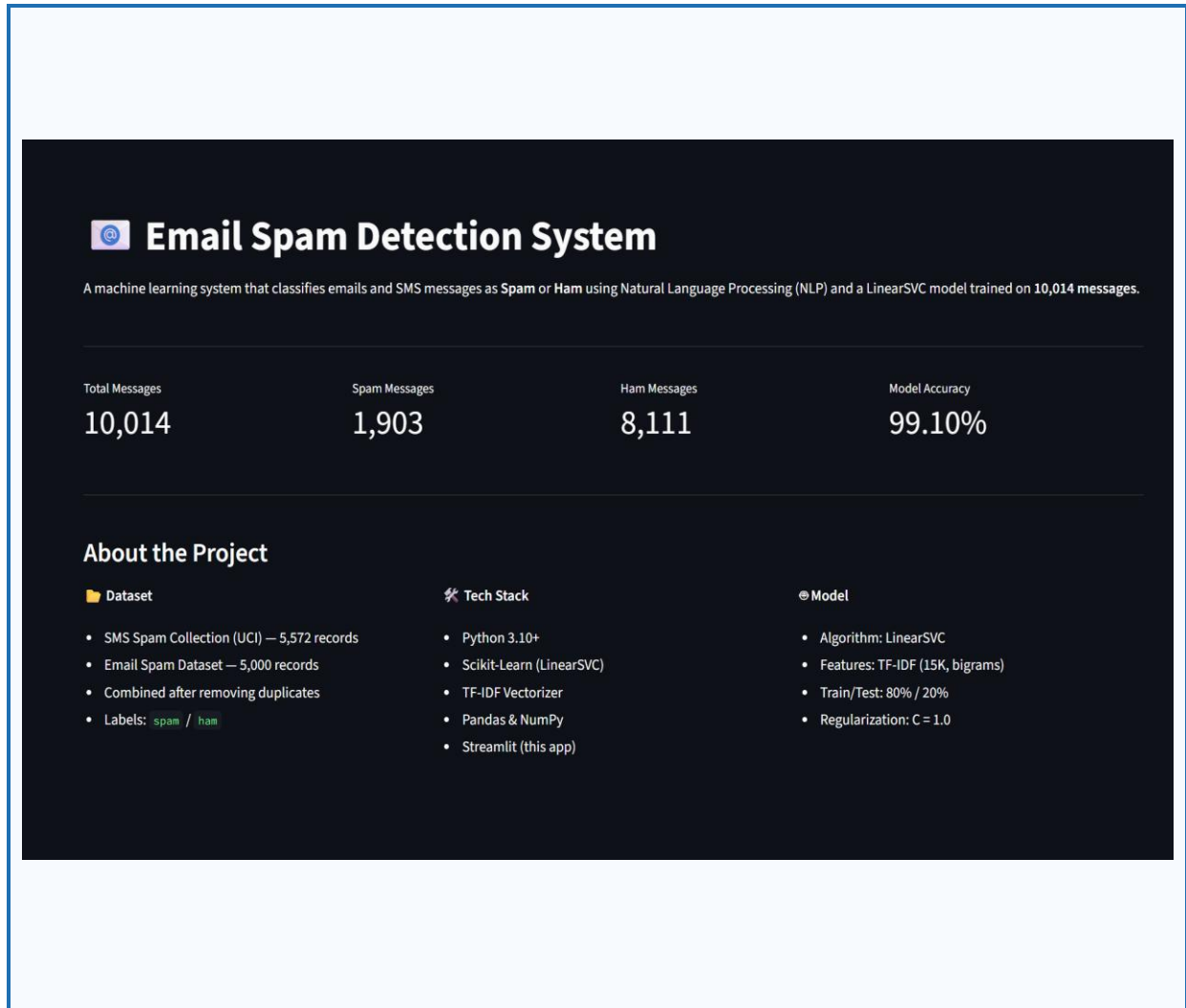


Figure 15.1: Streamlit Home Page

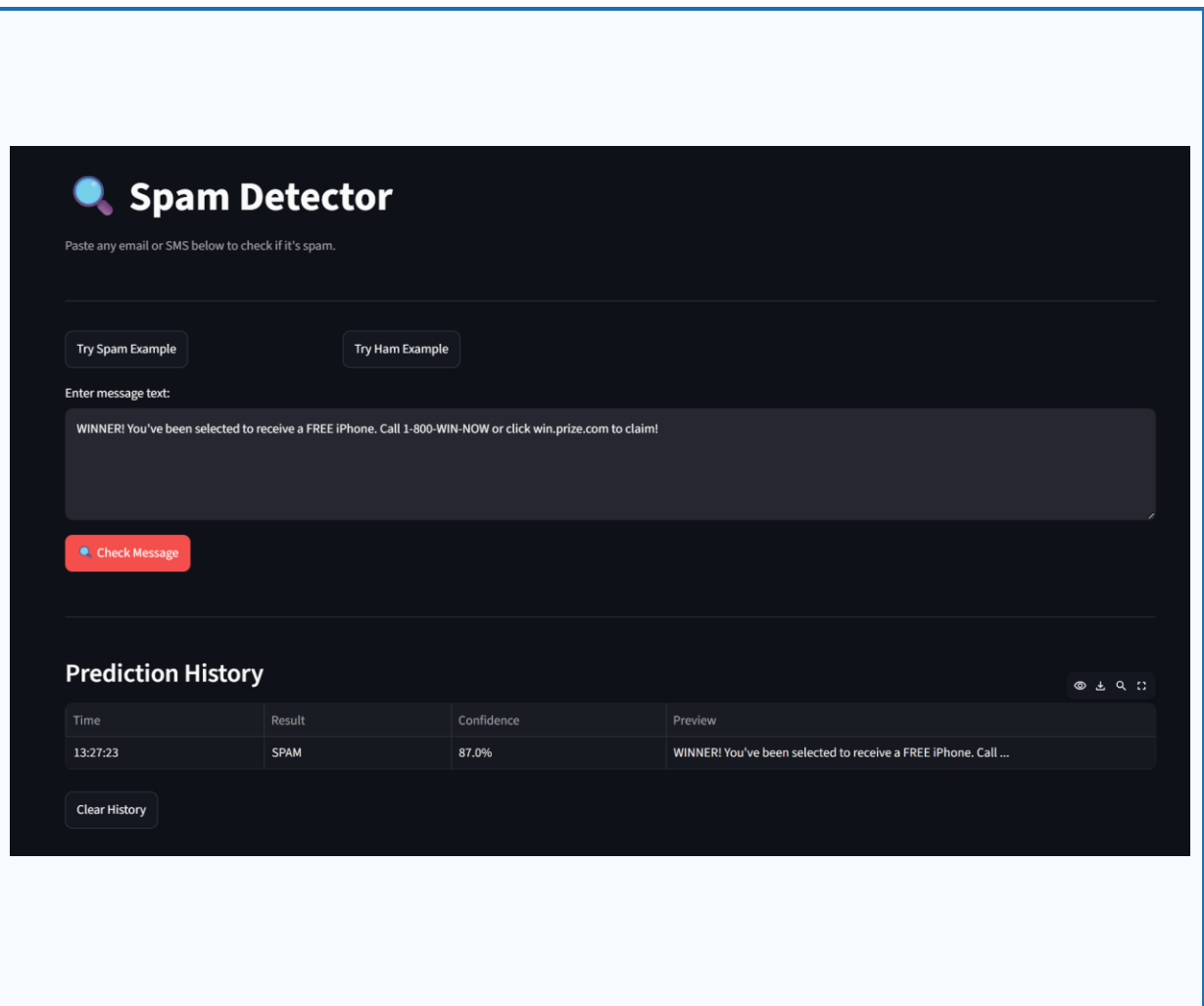
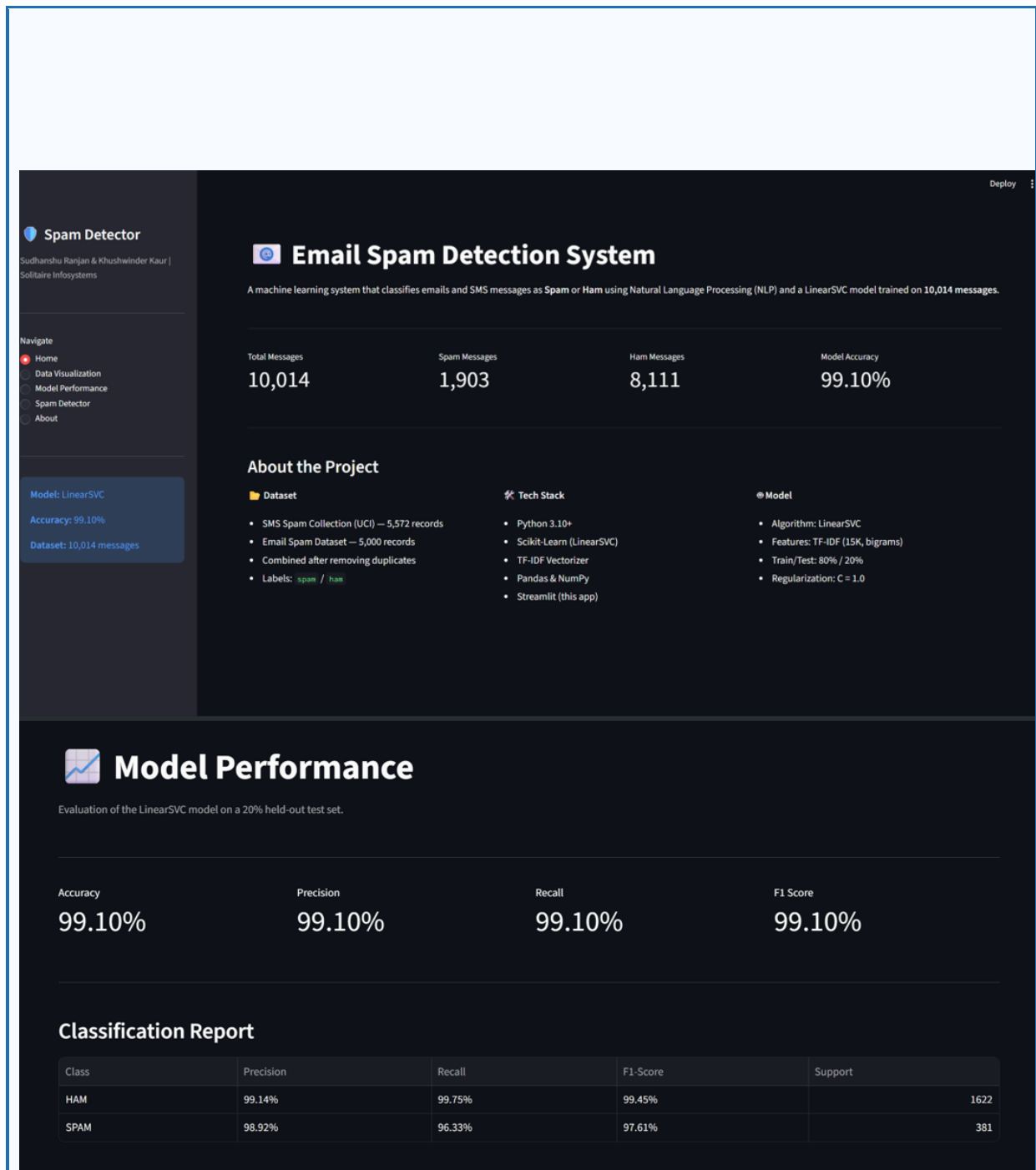


Figure 15.2: Streamlit Prediction Interface



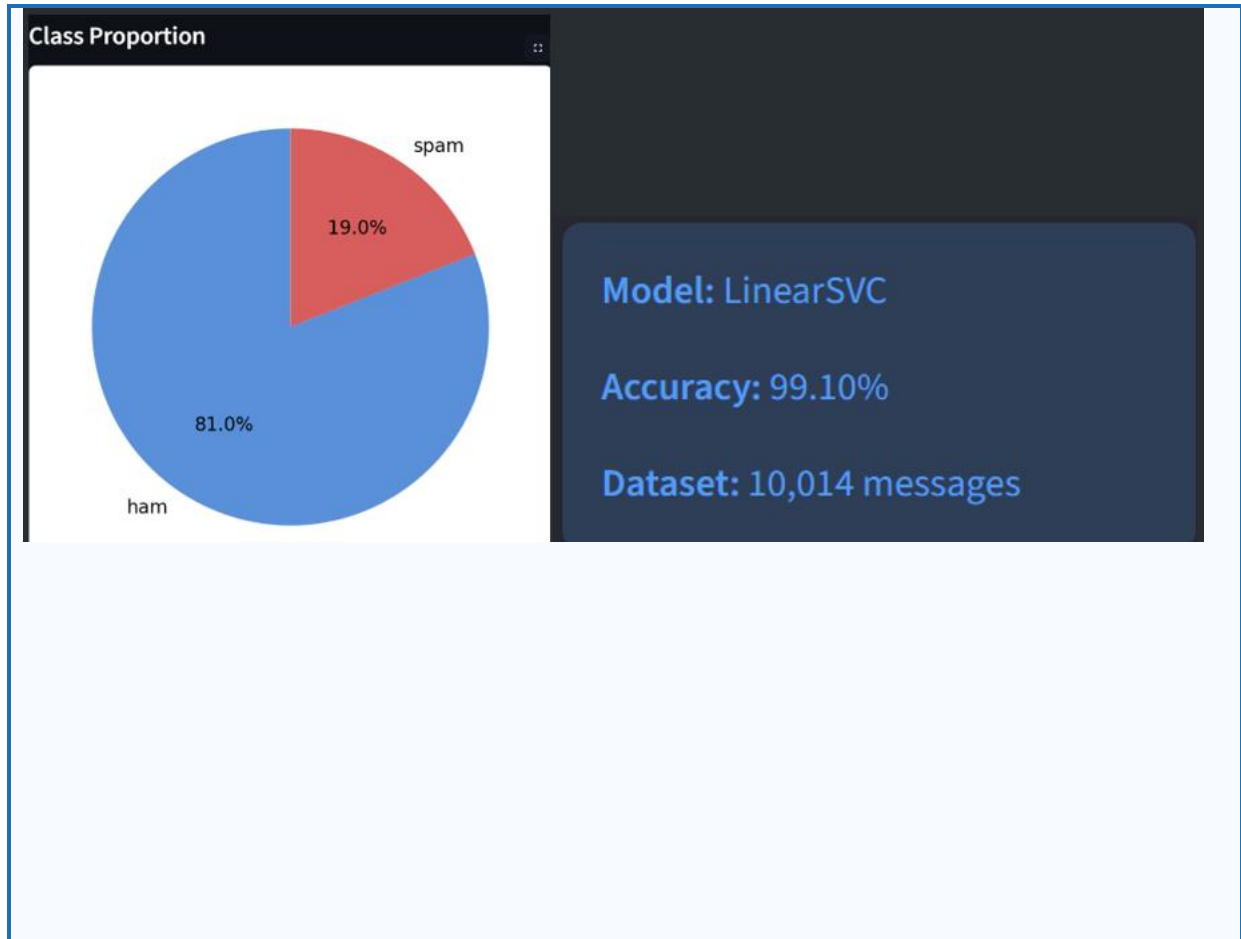


Figure 15.3: Streamlit Dashboard

15.3 Working of the Application

- The user enters email content into the text area.
- The application preprocesses the input text.
- The cleaned text is transformed into TF-IDF vectors.
- The saved LinearSVC model predicts the class label.
- The result is displayed to the user within a few milliseconds.

Chapter 16 - Testing

16.1 Introduction

Testing verifies that the application performs as expected under different conditions. Both functional and usability tests were considered to confirm that the model, interface, and saved pipeline work reliably.

16.2 Test Cases

Table 16.1: Functional Test Cases

Test ID	Test Scenario	Expected Result	Status
TC-01	Enter a spam email	Spam predicted	Pass
TC-02	Enter a legitimate email	Ham predicted	Pass
TC-03	Submit empty input	Warning message displayed	Pass
TC-04	Load application	Streamlit opens successfully	Pass
TC-05	Load trained model	Model loads without errors	Pass
TC-06	Display visualizations	Charts displayed correctly	Pass

16.3 Testing Summary

The application was tested using promotional messages, phishing examples, personal messages, and business communication samples. The model produced reliable predictions and the interface responded smoothly during testing.

Chapter 17 - Advantages & Limitations

17.1 Advantages

- High prediction accuracy for text-based spam detection.
- Automatic classification with minimal manual intervention.
- Real-time prediction through a simple web interface.
- Open-source implementation with low deployment cost.
- Scalable architecture that can be retrained with new data.
- Improves user protection against phishing and unwanted promotional messages.

17.2 Limitations

- The current model primarily supports English-language text.

- Performance depends on the quality and recency of training data.
- The system cannot analyze image-based spam without OCR support.
- Direct Gmail or Outlook integration is not included in the current version.
- New spam patterns may require periodic retraining.

Chapter 18 - Conclusion

The Email Spam Detection System using Machine Learning and Natural Language Processing was successfully designed, developed, and deployed during the internship at Solitaire Infosystems Pvt. Ltd. The project demonstrates the practical application of Artificial Intelligence in solving a real cybersecurity problem.

The project implemented a complete machine learning pipeline, including dataset collection, data cleaning, NLP preprocessing, TF-IDF feature extraction, LinearSVC model training, model evaluation, and Streamlit deployment. The use of NLP techniques improved the quality of text features and helped the classifier distinguish effectively between spam and legitimate messages.

The application provides a simple interface for instant email classification and shows how machine learning can automate repetitive tasks while improving security and reducing manual effort. The project also strengthened practical skills in Python, data preprocessing, text mining, supervised learning, web deployment, testing, and technical documentation.

Chapter 19 - Future Scope

The system can be enhanced in several directions to make it more robust and suitable for real-world deployment.

- Integrate with Gmail and Outlook APIs for real-time filtering.
- Support multilingual email classification.
- Implement deep learning models such as LSTM, GRU, or BERT.
- Detect image-based spam using OCR and image analysis.
- Add confidence scores and explainable AI features.
- Deploy the application on Streamlit Community Cloud, Render, AWS, or similar cloud platforms.
- Continuously retrain the model using newly collected email datasets.
- Develop a browser extension for automatic spam detection.

References

- [1] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, Data Mining: Practical Machine Learning Tools and Techniques, 4th ed. Morgan Kaufmann, 2016.
- [2] C. D. Manning, P. Raghavan, and H. Schutze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [4] Python Software Foundation, "Python Documentation." Accessed: 2026. [Online]. Available: <https://docs.python.org/3/>
- [5] The pandas development team, "pandas documentation." Accessed: 2026. [Online]. Available: <https://pandas.pydata.org/>
- [6] NumPy Developers, "NumPy documentation." Accessed: 2026. [Online]. Available: <https://numpy.org/>
- [7] NLTK Project, "Natural Language Toolkit Documentation." Accessed: 2026. [Online]. Available: <https://www.nltk.org/>
- [8] Streamlit, "Streamlit Documentation." Accessed: 2026. [Online]. Available: <https://streamlit.io/>
- [9] UCI Machine Learning Repository, "SMS Spam Collection Data Set." Accessed: 2026. [Online]. Available: <https://archive.ics.uci.edu/>
- [10] Kaggle, "Spam Collection Datasets." Accessed: 2026. [Online]. Available: <https://www.kaggle.com/>

Appendix

Appendix A - Software Used

- Python
- Jupyter Notebook
- Visual Studio Code
- Streamlit
- Git
- GitHub

Appendix B - Python Libraries

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Plotly
- NLTK
- Scikit-learn
- Joblib
- Streamlit

Appendix C - Screenshot Checklist

- Dataset preview
- Data cleaning output
- EDA graphs
- Word frequency chart
- TF-IDF processing
- Model training output
- Confusion matrix
- Classification report
- Streamlit home page
- Spam prediction page
- Dashboard page

- GitHub repository
- Folder structure
- Final application output

Appendix D - Core Prediction Pipeline

Code Block A.1: Example Scikit-learn Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC

model = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english")),
    ("classifier", LinearSVC())
])

model.fit(X_train, y_train)
prediction = model.predict([email_text])
```

Internship Learning Outcomes

- Python programming for machine learning.
- Natural Language Processing techniques.
- Data preprocessing and cleaning.
- Feature extraction using TF-IDF.
- Training and evaluating classification models.
- Developing interactive applications with Streamlit.
- Version control using Git and GitHub.
- Software testing, debugging, and technical reporting.